

✓ Loading the Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

✓ Importing the dataset

```
data = pd.read_csv("/content/AirPassengers.csv")
data.head()
```

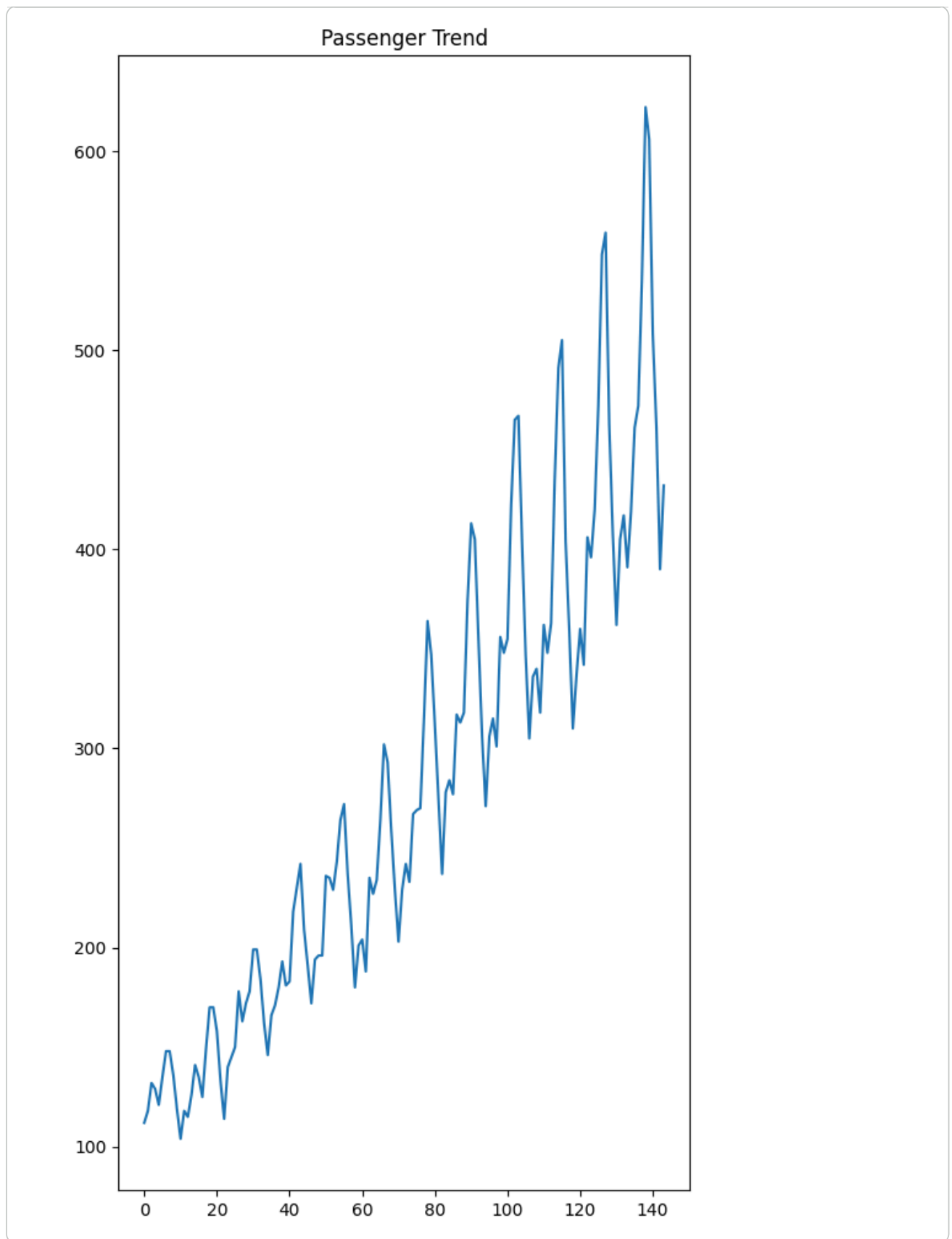
	Month	#Passengers
0	1949-01	112
1	1949-02	118
2	1949-03	132
3	1949-04	129
4	1949-05	121

```
data.isnull().sum()
```

	0
Month	0
#Passengers	0

dtype: int64

```
# Vizualing the number of passengers throughout the time.
plt.figure(figsize=(6, 12))
plt.plot(data['#Passengers'])
# plt.xlabel("Month of the year")
# plt.ylabel("Count of Passengers")
plt.title("Passenger Trend")
plt.show()
```



- The number of passengers increased with the time

✓ Data preprocessing

```
from sklearn.preprocessing import MinMaxScaler

# Extract passengers data as numpy array
```

```
passengers = data['#Passengers'].values.reshape(-1, 1)
print(type(passengers))
```

```
<class 'numpy.ndarray'>
```

```
scaler = MinMaxScaler()
scaled_data = scaler.fit_transform(passengers)
```

✓ Creating time sequence

✓ Creating sequence for past 12 months -> predict next month

```
def create_sequences(data, seq_length):
    X_list, y_list = [], []
    for i in range(len(data) - seq_length):
        X_list.append(data[i:i+seq_length])
        y_list.append(data[i+seq_length])
    return np.array(X_list), np.array(y_list)
seq_length = 24
X, y = create_sequences(scaled_data, seq_length)
```

✓ Print the shape of the x & y

```
print(X.shape)
print(y.shape)
```

```
(120, 24, 1)
(120, 1)
```

✓ Train-Test Split

```
ratio = int(0.8 * len(X))
X_train, X_test = X[:ratio], X[ratio:]
y_train, y_test = y[:ratio], y[ratio:]
```

✓ Simple RNN

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense
model_rnn = Sequential()
model_rnn.add(SimpleRNN(50, activation='tanh', input_shape=(seq_length,1)))
model_rnn.add(Dense(1))
model_rnn.compile(optimizer='adam', loss='mse')
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not p
super().__init__(**kwargs)
```

```
# Train the model
history_model = model_rnn.fit(X_train, y_train, epochs=50, batch_size=8, validation_split=0.1
```

```

11/11 ————— 0s 9ms/step - loss: 0.0014 - val_loss: 0.0039
Epoch 24/50
11/11 ————— 0s 9ms/step - loss: 0.0012 - val_loss: 0.0038
Epoch 25/50
11/11 ————— 0s 9ms/step - loss: 0.0012 - val_loss: 0.0032
Epoch 26/50
11/11 ————— 0s 8ms/step - loss: 0.0012 - val_loss: 0.0031
Epoch 27/50
11/11 ————— 0s 9ms/step - loss: 0.0015 - val_loss: 0.0060
Epoch 28/50
11/11 ————— 0s 8ms/step - loss: 0.0013 - val_loss: 0.0037
Epoch 29/50
11/11 ————— 0s 9ms/step - loss: 0.0011 - val_loss: 0.0029
Epoch 30/50
11/11 ————— 0s 8ms/step - loss: 0.0012 - val_loss: 0.0043
Epoch 31/50
11/11 ————— 0s 8ms/step - loss: 0.0011 - val_loss: 0.0028
Epoch 32/50
11/11 ————— 0s 9ms/step - loss: 0.0012 - val_loss: 0.0036
Epoch 33/50
11/11 ————— 0s 8ms/step - loss: 0.0014 - val_loss: 0.0030
Epoch 34/50
11/11 ————— 0s 8ms/step - loss: 0.0010 - val_loss: 0.0040
Epoch 35/50
11/11 ————— 0s 8ms/step - loss: 0.0010 - val_loss: 0.0034
Epoch 36/50
11/11 ————— 0s 9ms/step - loss: 9.7610e-04 - val_loss: 0.0028
Epoch 37/50
11/11 ————— 0s 7ms/step - loss: 9.3412e-04 - val_loss: 0.0026
Epoch 38/50
11/11 ————— 0s 8ms/step - loss: 0.0010 - val_loss: 0.0034
Epoch 39/50
11/11 ————— 0s 9ms/step - loss: 9.3869e-04 - val_loss: 0.0054
Epoch 40/50
11/11 ————— 0s 8ms/step - loss: 0.0012 - val_loss: 0.0027
Epoch 41/50
11/11 ————— 0s 9ms/step - loss: 9.8257e-04 - val_loss: 0.0024
Epoch 42/50
11/11 ————— 0s 12ms/step - loss: 9.7070e-04 - val_loss: 0.0024
Epoch 43/50
11/11 ————— 0s 12ms/step - loss: 9.3128e-04 - val_loss: 0.0022
Epoch 44/50
11/11 ————— 0s 10ms/step - loss: 9.3714e-04 - val_loss: 0.0036
Epoch 45/50
11/11 ————— 0s 10ms/step - loss: 8.5105e-04 - val_loss: 0.0022
Epoch 46/50
11/11 ————— 0s 10ms/step - loss: 9.1374e-04 - val_loss: 0.0024
Epoch 47/50
11/11 ————— 0s 11ms/step - loss: 9.5585e-04 - val_loss: 0.0037
Epoch 48/50
11/11 ————— 0s 10ms/step - loss: 8.3626e-04 - val_loss: 0.0021
Epoch 49/50
11/11 ————— 0s 8ms/step - loss: 8.3389e-04 - val_loss: 0.0033
Epoch 50/50
11/11 ————— 0s 9ms/step - loss: 8.6815e-04 - val_loss: 0.0042

```

LSTM

```

from tensorflow.keras.layers import LSTM
model_lstm = Sequential()
model_lstm.add(LSTM(50, input_shape=(seq_length, 1)))
model_lstm.add(Dense(1))
model_lstm.compile(optimizer='adam', loss='mse')

```

```

history_lstm = model_lstm.fit(X_train, y_train, epochs=50, batch_size=8, validation_split=0.1

```

```

Epoch 1/50
11/11  1s 30ms/step - loss: 0.0416 - val_loss: 0.0184
Epoch 2/50
11/11  0s 20ms/step - loss: 0.0102 - val_loss: 0.0203
Epoch 3/50
11/11  0s 17ms/step - loss: 0.0077 - val_loss: 0.0222
Epoch 4/50
11/11  0s 20ms/step - loss: 0.0069 - val_loss: 0.0175
Epoch 5/50
11/11  0s 16ms/step - loss: 0.0065 - val_loss: 0.0179
Epoch 6/50
11/11  0s 23ms/step - loss: 0.0064 - val_loss: 0.0180
Epoch 7/50
11/11  0s 23ms/step - loss: 0.0063 - val_loss: 0.0172
Epoch 8/50
11/11  0s 15ms/step - loss: 0.0063 - val_loss: 0.0171
Epoch 9/50
11/11  0s 12ms/step - loss: 0.0061 - val_loss: 0.0172
Epoch 10/50
11/11  0s 12ms/step - loss: 0.0061 - val_loss: 0.0173
Epoch 11/50
11/11  0s 12ms/step - loss: 0.0058 - val_loss: 0.0169
Epoch 12/50
11/11  0s 13ms/step - loss: 0.0063 - val_loss: 0.0168
Epoch 13/50
11/11  0s 12ms/step - loss: 0.0061 - val_loss: 0.0165
Epoch 14/50
11/11  0s 11ms/step - loss: 0.0060 - val_loss: 0.0164
Epoch 15/50
11/11  0s 11ms/step - loss: 0.0059 - val_loss: 0.0163
Epoch 16/50
11/11  0s 12ms/step - loss: 0.0057 - val_loss: 0.0164
Epoch 17/50
11/11  0s 11ms/step - loss: 0.0056 - val_loss: 0.0162
Epoch 18/50
11/11  0s 12ms/step - loss: 0.0056 - val_loss: 0.0165
Epoch 19/50
11/11  0s 13ms/step - loss: 0.0063 - val_loss: 0.0170
Epoch 20/50
11/11  0s 11ms/step - loss: 0.0064 - val_loss: 0.0157
Epoch 21/50
11/11  0s 11ms/step - loss: 0.0054 - val_loss: 0.0156
Epoch 22/50
11/11  0s 11ms/step - loss: 0.0058 - val_loss: 0.0158
Epoch 23/50
11/11  0s 11ms/step - loss: 0.0056 - val_loss: 0.0155
Epoch 24/50
11/11  0s 10ms/step - loss: 0.0055 - val_loss: 0.0151
Epoch 25/50
11/11  0s 11ms/step - loss: 0.0061 - val_loss: 0.0155
Epoch 26/50
11/11  0s 12ms/step - loss: 0.0060 - val_loss: 0.0160
Epoch 27/50
11/11  0s 11ms/step - loss: 0.0058 - val_loss: 0.0148
Epoch 28/50
11/11  0s 11ms/step - loss: 0.0052 - val_loss: 0.0146
Epoch 29/50
11/11  0s 11ms/step - loss: 0.0053 - val_loss: 0.0147

```

✓ GRU

```

from tensorflow.keras.layers import GRU
model_gru = Sequential()
model_gru.add(GRU(50, input_shape=(seq_length, 1)))
model_gru.add(Dense(1))
model_gru.compile(optimizer='adam', loss='mse')

```

```
history_gru = model_gru.fit(X_train, y_train, epochs=50, batch_size=8, validation_split=0.1)
```

```
Epoch 22/50
11/11  0s 13ms/step - loss: 0.0052 - val_loss: 0.0169
Epoch 23/50
11/11  0s 12ms/step - loss: 0.0052 - val_loss: 0.0147
Epoch 24/50
11/11  0s 13ms/step - loss: 0.0052 - val_loss: 0.0150
Epoch 25/50
11/11  0s 13ms/step - loss: 0.0049 - val_loss: 0.0138
Epoch 26/50
11/11  0s 13ms/step - loss: 0.0049 - val_loss: 0.0137
Epoch 27/50
11/11  0s 12ms/step - loss: 0.0049 - val_loss: 0.0137
Epoch 28/50
11/11  0s 13ms/step - loss: 0.0046 - val_loss: 0.0127
Epoch 29/50
11/11  0s 13ms/step - loss: 0.0044 - val_loss: 0.0124
Epoch 30/50
11/11  0s 14ms/step - loss: 0.0043 - val_loss: 0.0120
Epoch 31/50
11/11  0s 13ms/step - loss: 0.0042 - val_loss: 0.0117
Epoch 32/50
11/11  0s 12ms/step - loss: 0.0040 - val_loss: 0.0113
Epoch 33/50
11/11  0s 12ms/step - loss: 0.0042 - val_loss: 0.0110
Epoch 34/50
11/11  0s 13ms/step - loss: 0.0045 - val_loss: 0.0122
Epoch 35/50
11/11  0s 13ms/step - loss: 0.0039 - val_loss: 0.0105
Epoch 36/50
11/11  0s 13ms/step - loss: 0.0038 - val_loss: 0.0107
Epoch 37/50
11/11  0s 13ms/step - loss: 0.0036 - val_loss: 0.0099
Epoch 38/50
11/11  0s 12ms/step - loss: 0.0038 - val_loss: 0.0103
Epoch 39/50
11/11  0s 12ms/step - loss: 0.0036 - val_loss: 0.0098
Epoch 40/50
11/11  0s 13ms/step - loss: 0.0033 - val_loss: 0.0100
Epoch 41/50
11/11  0s 13ms/step - loss: 0.0034 - val_loss: 0.0090
Epoch 42/50
11/11  0s 12ms/step - loss: 0.0033 - val_loss: 0.0092
Epoch 43/50
11/11  0s 12ms/step - loss: 0.0034 - val_loss: 0.0123
Epoch 44/50
11/11  0s 14ms/step - loss: 0.0034 - val_loss: 0.0087
Epoch 45/50
11/11  0s 14ms/step - loss: 0.0030 - val_loss: 0.0094
Epoch 46/50
11/11  0s 16ms/step - loss: 0.0034 - val_loss: 0.0110
Epoch 47/50
11/11  0s 14ms/step - loss: 0.0031 - val_loss: 0.0082
Epoch 48/50
11/11  0s 15ms/step - loss: 0.0034 - val_loss: 0.0082
Epoch 49/50
11/11  0s 16ms/step - loss: 0.0034 - val_loss: 0.0110
Epoch 50/50
11/11  0s 12ms/step - loss: 0.0031 - val_loss: 0.0078
```

✓ Evaluation

```
from sklearn.metrics import mean_squared_error
import math

def evaluate(model):
```

```

pred = model.predict(X_test)
pred = scaler.inverse_transform(pred)
actual = scaler.inverse_transform(y_test)

rmse = math.sqrt(mean_squared_error(actual, pred))
return rmse

```

```

performance_rnn = evaluate(model_rnn)
performance_lstm = evaluate(model_lstm)
performance_gru = evaluate(model_gru)

```

```

1/1 ————— 0s 116ms/step
WARNING:tensorflow:5 out of the last 5 calls to <function TensorFlowTrainer.make_predict_funct
1/1 ————— 0s 131ms/step
WARNING:tensorflow:6 out of the last 6 calls to <function TensorFlowTrainer.make_predict_funct
1/1 ————— 0s 144ms/step

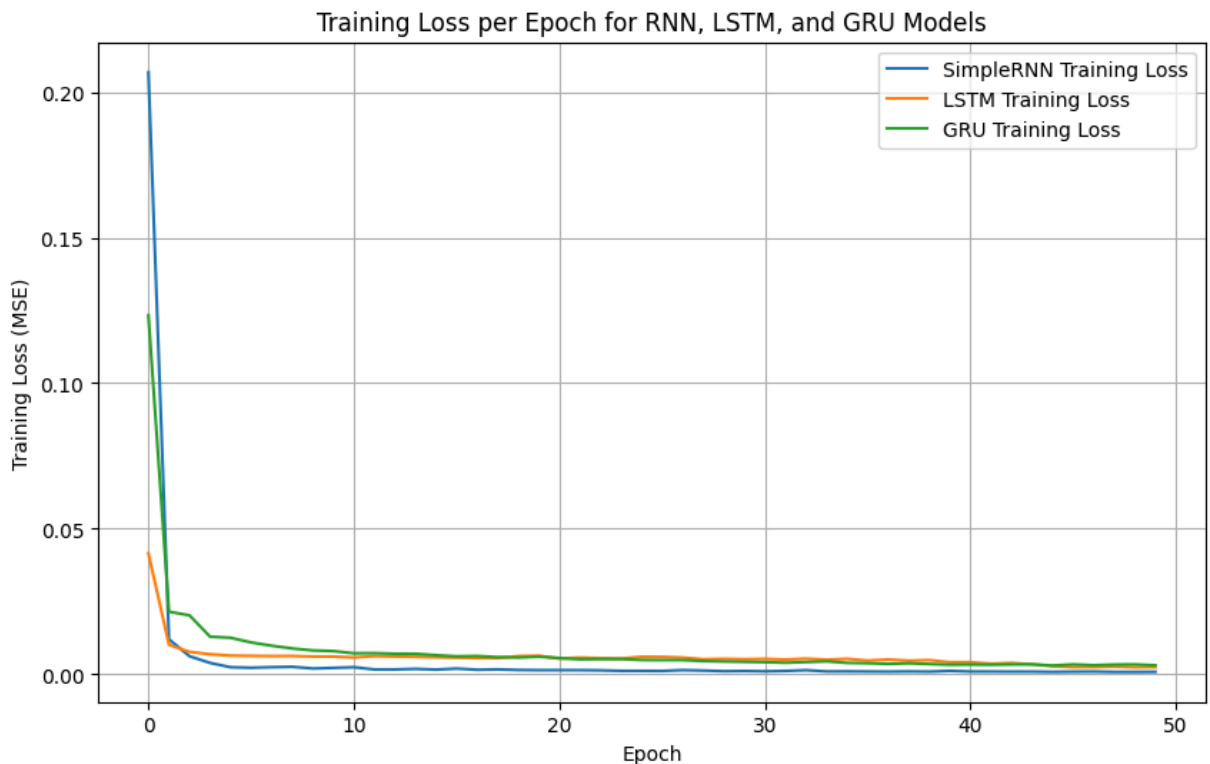
```

✓ Vizualization of the loss

```

plt.figure(figsize=(10, 6))
plt.plot(history_model.history['loss'], label='SimpleRNN Training Loss')
plt.plot(history_lstm.history['loss'], label='LSTM Training Loss')
plt.plot(history_gru.history['loss'], label='GRU Training Loss')
plt.title('Training Loss per Epoch for RNN, LSTM, and GRU Models')
plt.xlabel('Epoch')
plt.ylabel('Training Loss (MSE)')
plt.legend()
plt.grid(True)
plt.show()

```

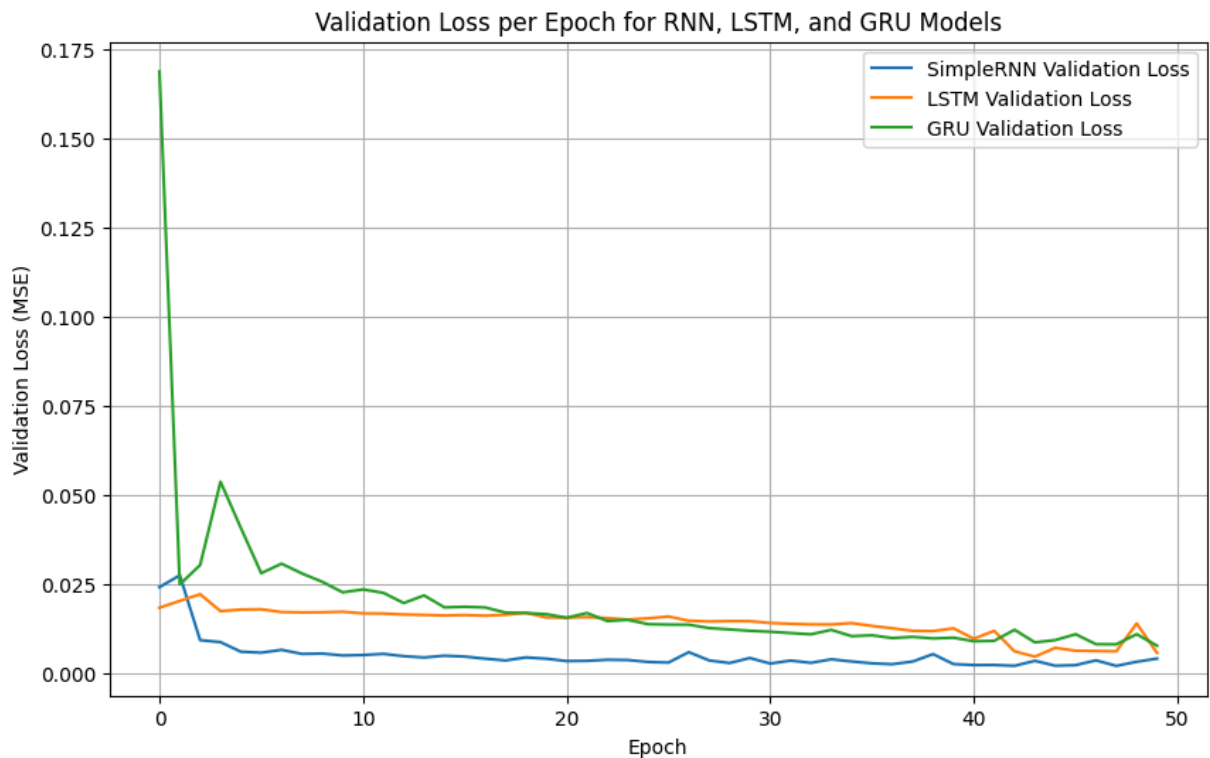


```

plt.figure(figsize=(10, 6))
plt.plot(history_model.history['val_loss'], label='SimpleRNN Validation Loss')
plt.plot(history_lstm.history['val_loss'], label='LSTM Validation Loss')
plt.plot(history_gru.history['val_loss'], label='GRU Validation Loss')

```

```
plt.title('Validation Loss per Epoch for RNN, LSTM, and GRU Models')
plt.xlabel('Epoch')
plt.ylabel('Validation Loss (MSE)')
plt.legend()
plt.grid(True)
plt.show()
```



✓ Knoledge Check

1. What is sequential data?

- Sequential data is any data where the order of values carries meaning. Unlike a spreadsheet of independent customer records, where you could shuffle the rows without losing information, sequential data loses its meaning the moment you rearrange it.

2. What is the vanishing gradient problem?

- Vanishing gradients occur when gradients become extremely small during backpropagation, causing early layers to learn very slowly or stop learning.

3. Why is LSTM better than Simple RNN?

- LSTM is preffered over RNN because LSTM stores the memory in the memory gate and also forget certain points which are irrelevant for model. Also LSTM performs better over large data

4. What is the role of timestep in RNN?

- In a Recurrent Neural Network (RNN), the timestep represents one position in a sequence — essentially, one unit of time in which the network processes part of the input.

✓ 5. What is the difference between LSTM and GRU

- LSTM : Memory cells with input, forget and output gates.
GRU: Combines input and forget gates into update gate; fewer parameters
- LSTM: Slower due to multiple gates and memory operation
GRU: Faster than LSTM.